

```
In [ ]: #Factorial number:
```

```
In [15]: #Using function:  
def fact(n):  
    f = 1  
    for i in range(1, n+1):  
        f = f*i  
    return f  
  
x = 6  
result = fact(x)  
print(result)
```

720

```
In [ ]: #Recursion : Calling function from itself
```

```
In [1]: def wish():  
    print('hello')  
    print('hi')  
    wish()
```

hello
hi

```
In [3]: def wish():  
    print('hello')  
    print('hi')  
    wish()  
    wish()
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```

-----
RecursionError                                Traceback (most recent call last)
Cell In[3], line 5
      3     print('hi')
      4     wish()
----> 5 wish()

Cell In[3], line 4, in wish()
      2     print('hello')
      3     print('hi')
----> 4 wish()

Cell In[3], line 4, in wish()
      2     print('hello')
      3     print('hi')
----> 4 wish()

[... skipping similar frames: wish at line 4 (2972 times)]

Cell In[3], line 4, in wish()
      2     print('hello')
      3     print('hi')
----> 4 wish()

Cell In[3], line 2, in wish()
      1     def wish():
----> 2         print('hello')
      3         print('hi')
      4         wish()

File ~\anaconda3\Lib\site-packages\ipykernel\iostream.py:664, in OutStream.write(self, string)
    655 def write(self, string: str) -> Optional[int]: # type:ignore[override]
    656     """Write to current stream after encoding if necessary
    657
    658     Returns
    659     (...)
    662
    663     """
--> 664     parent = self.parent_header
    666     if not isinstance(string, str):
    667         msg = f"write() argument must be str, not {type(string)}" # type:ignore[unreachable]

```


RecursionError: maximum recursion depth exceeded

```
In [6]: import sys  
sys.getrecursionlimit()
```

Out[6]: 3000

```
In [8]: import sys  
sys.setrecursionlimit(200)  
print(sys.getrecursionlimit())
```

200

```
In [9]: import sys  
sys.getrecursionlimit()
```

Out[9]: 200

```
In [10]: def wish():  
    print('hello')  
    print('hi')  
    wish()  
wish()
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

hello
hi
hello
hi
hello
hi
hello
hi
hello
hi
hello
hi
hello
hi

```

-----
RecursionError                                Traceback (most recent call last)
Cell In[10], line 5
      3     print('hi')
      4     wish()
----> 5 wish()

Cell In[10], line 4, in wish()
      2 print('hello')
      3 print('hi')
----> 4 wish()

Cell In[10], line 4, in wish()
      2 print('hello')
      3 print('hi')
----> 4 wish()

[... skipping similar frames: wish at line 4 (172 times)]

Cell In[10], line 4, in wish()
      2 print('hello')
      3 print('hi')
----> 4 wish()

Cell In[10], line 2, in wish()
      1 def wish():
----> 2     print('hello')
      3     print('hi')
      4     wish()

File ~\anaconda3\Lib\site-packages\ipykernel\iostream.py:664, in OutStream.write(self, string)
    655 def write(self, string: str) -> Optional[int]: # type:ignore[override]
    656     """Write to current stream after encoding if necessary
    657
    658     Returns
    659     (...)
    662
    663     """
--> 664     parent = self.parent_header
    666     if not isinstance(string, str):
    667         msg = f"write() argument must be str, not {type(string)}" # type:ignore[unreachable]

```

RecursionError: maximum recursion depth exceeded

```
In [11]: import sys
sys.setrecursionlimit(150)
print(sys.getrecursionlimit())

i = 0

def wish():
    global i
    i += 1
    print("hello", i)
    wish()
wish()
```

150

hello 1
hello 2
hello 3
hello 4
hello 5
hello 6
hello 7
hello 8
hello 9
hello 10
hello 11
hello 12
hello 13
hello 14
hello 15
hello 16
hello 17
hello 18
hello 19
hello 20
hello 21
hello 22
hello 23
hello 24
hello 25
hello 26
hello 27
hello 28
hello 29
hello 30
hello 31
hello 32
hello 33
hello 34
hello 35
hello 36
hello 37
hello 38
hello 39
hello 40
hello 41

hello 42
hello 43
hello 44
hello 45
hello 46
hello 47
hello 48
hello 49
hello 50
hello 51
hello 52
hello 53
hello 54
hello 55
hello 56
hello 57
hello 58
hello 59
hello 60
hello 61
hello 62
hello 63
hello 64
hello 65
hello 66
hello 67
hello 68
hello 69
hello 70
hello 71
hello 72
hello 73
hello 74
hello 75
hello 76
hello 77
hello 78
hello 79
hello 80
hello 81
hello 82
hello 83

hello 84
hello 85
hello 86
hello 87
hello 88
hello 89
hello 90
hello 91
hello 92
hello 93
hello 94
hello 95
hello 96
hello 97
hello 98
hello 99
hello 100
hello 101
hello 102
hello 103
hello 104
hello 105
hello 106
hello 107
hello 108
hello 109
hello 110
hello 111
hello 112
hello 113
hello 114
hello 115
hello 116
hello 117
hello 118
hello 119
hello 120
hello 121
hello 122
hello 123
hello 124
hello 125

```

-----
RecursionError                                Traceback (most recent call last)
Cell In[11], line 12
     10     print("hello", i)
     11     wish()
--> 12 wish()

Cell In[11], line 11, in wish()
     9 i +=1
    10 print("hello", i)
--> 11 wish()

Cell In[11], line 11, in wish()
     9 i +=1
    10 print("hello", i)
--> 11 wish()

[... skipping similar frames: wish at line 11 (122 times)]

Cell In[11], line 11, in wish()
     9 i +=1
    10 print("hello", i)
--> 11 wish()

Cell In[11], line 10, in wish()
     8 global i
     9 i +=1
--> 10 print("hello", i)
    11 wish()

File ~\anaconda3\Lib\site-packages\ipykernel\iostream.py:664, in OutStream.write(self, string)
    655 def write(self, string: str) -> Optional[int]: # type:ignore[override]
    656     """Write to current stream after encoding if necessary
    657
    658     Returns
    659     (...)
    662
    663     """
--> 664     parent = self.parent_header
    666     if not isinstance(string, str):
    667         msg = f"write() argument must be str, not {type(string)}" # type:ignore[unreachable]

```


RecursionError: maximum recursion depth exceeded

In [16]: *#Factorial using recursion :*

```
def fact(n):  
    if n == 0:  
        return 1  
    return n * fact(n-1)  
result = fact(5)  
result
```

Out[16]: 120

In []: *#Anonymous Function/Lambda:*

In [18]: *#Without variable using function:*

```
def square(a):  
    return a*a  
square(5)
```

Out[18]: 25

In [19]: *#With variable using function:*

```
def square(a):  
    return a*a  
result = square(5)  
print(result)
```

25

In [20]: *#Using lambda:*

```
f = lambda a : a*a  
result = f(5)  
result
```

Out[20]: 25

In [21]: *#Add and sub 2 numbers:*

```
f = lambda a,b : a+b  
f1 = lambda a,b : a-b
```

```
result = f(1,4)
result1 = f1(4,1)

print(result)
print(result1)
```

5
3

In [22]: *#Add, sub and multiply 2 numbers:*

```
f = lambda a,b : a+b
f1 = lambda a,b : a-b
f2 = lambda a,b : a*b
```

```
result = f(1,4)
result1 = f1(4,1)
result2 = f2(4,1)
```

```
print(result)
print(result1)
print(result2)
```

5
3
4

In [23]:

```
nums = [3,2,6,8,4,6,2,9]
evens = list(filter(is_even, nums))
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[23], line 2
      1 nums = [3,2,6,8,4,6,2,9]
----> 2 evens = list(filter(is_even, nums))

NameError: name 'is_even' is not defined
```

In [25]: *#Filter all even numbers in the list*

```
def is_even(n):
    return n%2 == 0

nums = [3,2,6,8,4,6,2,9]
```

```
evens = list(filter(is_even, nums))
print(evens)
```

[2, 6, 8, 4, 6, 2]

In [26]: *#Filter all odd numbers in the List*

```
def is_odd(n):
    return n%2 != 0

nums = [3,2,6,8,4,6,2,9]
odd = list(filter(is_odd, nums))
print(odd)
```

[3, 9]

In [28]: *#Using lambda, print even numbers:*

```
nums = [3,2,6,8,4,6,2,9]
evens = list(filter(lambda n : n%2 == 0, nums))
print(evens)
```

[2, 6, 8, 4, 6, 2]

In [29]: *#Using lambda, print even numbers:*

```
nums = [3,2,6,8,4,6,2,9]
odd = list(filter(lambda n : n%2 != 0, nums))
print(odd)
```

[3, 9]

In [30]: *#Combine both even and odd*

```
nums = [3,2,6,8,4,6,2,9,34,77,120]
evens = list(filter(lambda n : n%2 == 0, nums))
odd = list(filter(lambda n : n%2 != 0, nums))
print(evens)
print(odd)
```

[2, 6, 8, 4, 6, 2, 34, 120]

[3, 9, 77]

In [32]: **def** is_even(n):

```
    return n%2 == 0
```

```
def update(n):
```

```
    return n+2
```

```
nums = [3,2,6,8,4,6,2,9]
evens = list(filter(is_even, nums))
double = list(map(update, evens))
print(evens)
print(double)
```

```
[2, 6, 8, 4, 6, 2]
[4, 8, 10, 6, 8, 4]
```

```
In [33]: #Using Lambda
def is_even(n):
    return n%2 == 0

nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))
double = list(map(lambda n : n+2, evens))
print(evens)
print(double)
```

```
[2, 6, 8, 4, 6, 2]
[4, 8, 10, 6, 8, 4]
```

```
In [34]: #Using Lambda
def is_even(n):
    return n%2 == 0

nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))
double = list(map(lambda n : n*2, evens))
print(evens)
print(double)
```

```
[2, 6, 8, 4, 6, 2]
[4, 12, 16, 8, 12, 4]
```

```
In [35]: #Using Lambda
def is_even(n):
    return n%2 == 0

nums = [3,2,6,8,4,6,2,9]
```

```
evens = list(filter(is_even, nums))
double = list(map(lambda n : n+2, evens))
double1 = list(map(lambda n : n*2, evens))
double2 = list(map(lambda n : n-2, evens))
print(evens)
print(double)
print(double1)
print(double2)
```

```
[2, 6, 8, 4, 6, 2]
[4, 8, 10, 6, 8, 4]
[4, 12, 16, 8, 12, 4]
[0, 4, 6, 2, 4, 0]
```

```
In [37]: from functools import reduce
def add_all(a,b):
    return a+b
evens = list(filter(is_even, nums))
double = list(map(lambda n : n*2, evens))
sums = reduce(add_all, double)
sums
print(sums)
```

56

```
In [38]: from functools import reduce
def add_all(a,b):
    return a+b
evens = list(filter(is_even, nums))
double = list(map(lambda n : n+2, evens))
sums = reduce(add_all, double)
sums
print(sums)
```

40

```
In [39]: from functools import reduce
def add_all(a,b):
    return a+b
evens = list(filter(is_even, nums))
double = list(map(lambda n : n-2, evens))
sums = reduce(add_all, double)
```

```
sums  
print(sums)
```

16

```
In [40]: #Using Lambda:  
from functools import reduce  
evens = list(filter(is_even, nums))  
double = list(map(lambda n : n*2, evens))  
sums = reduce(lambda a,b : a+b, double)  
print(evens)  
print(double)  
print(sums)
```

```
[2, 6, 8, 4, 6, 2]  
[4, 12, 16, 8, 12, 4]  
56
```

In []: